ARBITRAGE TRADING TERMINAL

Architecture / Technical Specification / Development Prompt

Spot ↔ Futures Spread Arbitrage

Version 1.0 ▪ 2026-03-15
Exchanges: Binance, Bybit, Gate.io, OKX

① АРХИТЕКТУРА СИСТЕМЫ

1.1 Обзор

Terminal-based Python приложение, которое читает арбитражные сигналы из **Redis**, позволяет оператору подтвердить вход, открывает парные лимитные ордера (спот + фьючерс шорт), мониторит исполнение, показывает **live PnL**, и закрывает по маркету по команде оператора.

1.2 Компоненты

Компонент	Ответственность	Файл
config.py	Все настройки: API ключи, Redis, параметры торговли	config.py
SignalListener	Подписка на Redis pub/sub, парсинг JSON и CSV форматов	signal_listener.py
ExchangeManager	CCXT-обёртка для 4 бирж (spot + futures), округление, ордера	exchange_manager.py
TradeExecutor	Расчёт цен, открытие/мониторинг/закрытие сделок	trade_executor.py
Terminal UI (main)	Rich TUI: отображение, ввод, состояния, логирование	main.py
test_signal.py	Тестовая отправка сигналов в Redis	test_signal.py

1.3 Поток данных

```
01  Redis ch:spread_signals  ──▶  SignalListener (парсит JSON/CSV)
02  SignalListener  ──yield Signal──▶  Terminal UI
03  Terminal UI  ──показ + вопрос Y/N──▶  Оператор
04  Оператор Y  ──▶  TradeExecutor.open_trade()
05  TradeExecutor  ──2 threads──▶  ExchangeManager (limit buy + limit sell)
06  ExchangeManager  ──ccxt──▶  Binance / Bybit / Gate / OKX API
07  Poll loop (1s)  ──▶  обновление статуса ордеров  ──▶  Rich Live display
08  PnL loop (1-2s)  ──▶  fetch tickers  ──▶  compute PnL  ──▶  Live display
09  Оператор 'close'  ──▶  TradeExecutor.close_trade()  ──▶  2x market orders
10  Лог-файл  ──▶  logs/trade_SYMBOL_TIMESTAMP.log
```

1.4 Машина состояний

Состояние	Описание	Переход
IDLE	Ожидание сигнала из Redis	Сигнал пришёл → SIGNAL_RECEIVED
SIGNAL_RECEIVED	Показ инфо, вопрос Y/N	Y → OPENING / N → IDLE
OPENING	Отправка лимиток на 2 биржи	Ордера отправлены → MONITORING_FILL
MONITORING_FILL	Поллинг статуса каждую секунду	Оба filled → POSITION_OPEN
POSITION_OPEN	Live PnL, текущий спред	'close' → CLOSING
CLOSING	Маркет-ордера на закрытие	Исполнено → CLOSED
CLOSED	Итог, лог-файл	Авто → IDLE

② ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Форматы сигналов

JSON формат (из Redis ch:spread_signals)

Поле	Тип	Описание	Required
symbol	string	Торговая пара (BTCUSDT, THEUSDT)	да
direction	string	Направление арбитража (A/B)	нет
buy_exchange	string	Биржа покупки: binance/bybit/gate/okx	да
buy_market	string	spot или futures	нет, default spot
buy_ask	string/num	Цена ask на бирже покупки	да
buy_ask_qty	string/num	Объём на этом ask	нет
sell_exchange	string	Биржа продажи (short)	да
sell_market	string	spot или futures	нет, default futures
sell_bid	string/num	Цена bid на бирже продажи	да
sell_bid_qty	string/num	Объём на этом bid	нет
spread_pct	float	Коэффициент спреда (1.028 = 2.8%)	да
ts_signal	int	Unix timestamp в миллисекундах	да

CSV формат (альтернативный)

`buy_exchange,sell_exchange.SYMBOL,buy_ask,sell_bid,spread_pct,ts_signal`

Пример: `binance,gate.THEUSDT,0.2338,0.2362,1.0265,1773590375286`

Парсер должен поддерживать оба формата: сначала пытаться JSON, при ошибке — CSV.

2.2 Логика расчёта цен входа (КРИТИЧЕСКИ ВАЖНО)

> **Главный принцип:** Спред между нашими лимитками должен быть меньше рыночного на **SPREAD_REDUCTION пп** (процентных пунктов). Это снижение распределяется поровну: половина съедается на стороне покупки, половина на стороне продажи.

Настраиваемый параметр: SPREAD_REDUCTION = 0.30 (по умолчанию 0.30пп, т.е. 0.15пп с каждой стороны)

Оператор может менять этот параметр в **config** или при запуске.

Формулы

```
market_spread_pct = (sell_bid / buy_ask - 1) * 100

target_spread_pct = market_spread_pct - SPREAD_REDUCTION
half_reduction     = SPREAD_REDUCTION / 2

// Покупка спота: поднимаем цену (платим больше)
adjusted_buy = buy_ask * (1 + half_reduction / 100)

// Продажа фьючерса: опускаем цену (получаем меньше)
adjusted_sell = sell_bid * (1 - half_reduction / 100)

// Проверка:
actual_spread = (adjusted_sell / adjusted_buy - 1) * 100
// actual_spread ≈ market_spread_pct - SPREAD_REDUCTION
```

Пример

Параметр	Значение	Пояснение
buy_ask	0.2338	Рыночный ask на Binance spot
sell_bid	0.2362	Рыночный bid на Gate futures
market_spread	1.026%	(0.2362/0.2338 - 1) * 100
SPREAD_REDUCTION	0.30пп	Настраиваемый параметр
half_reduction	0.15пп	0.30 / 2
adjusted_buy	0.23415	0.2338 * 1.0015 = платим больше
adjusted_sell	0.23585	0.2362 * 0.9985 = получаем меньше
result_spread	0.726%	≈ 1.026 - 0.30 = 0.726% ✔

> ❗ **ВАЖНО:** Сдвиг цены пропорционален самой цене (умножение на 1±0.0015), а не абсолютный сдвиг. Это обеспечивает корректное уменьшение спреда в процентных пунктах независимо от цены актива.

2.3 Жизненный цикл ордера

Открытие (лимитные ордера)

- Оба ордера отправляются параллельно через threading (не последовательно!)
- BUY leg: limit buy на spot-бирже по adjusted_buy
- SELL leg: limit sell на futures-бирже по adjusted_sell (открытие short)
- Количество: position_size_usdt / adjusted_price, округлённое по правилам биржи
- Фактические USDT могут отличаться из-за округления — это нормально
- Fees и slippage не учитываем при расчёте размера

Мониторинг заполнения

- Poll каждую секунду (ORDER_POLL_INTERVAL): fetch_order() для обоих ног
- Показывать: order_id, status, filled_amount, avg_price, intended_amount
- Статусы: pending → open → closed / canceled / error
- Rich Live обновляет таблицу в реальном времени
- Timeout 5 мин — переход к PnL монитору даже если не все заполнились

Live PnL

- Fetch ticker с обеих бирж каждые 1-2 секунды
- PnL покупки (long spot) = (current_bid - entry_price) * filled_amount
- PnL продажи (short futures) = (entry_price - current_ask) * filled_amount
- Total PnL = buy_pnl + sell_pnl
- Текущий спред vs спред входа — показывать оба
- Длительность позиции

Закрытие (маркет ордера)

- Оператор нажимает 'close' или Ctrl+C
- BUY leg: market sell (закрыть long спот)
- SELL leg: market buy (закрыть short фьючерс)
- Параллельно через threads
- После закрытия — финальный PnL + лог-файл

2.4 Нюансы бирж

Биржа	CCXT ID	Нюансы
Binance	binance	Spot: BASE/QUOTE. Futures (USDM): BASE/QUOTE:QUOTE. Для шорта нужен margin mode (cross/isolated). Проверить positionSide.
Bybit	bybit	Unified account. Futures: BASE/QUOTE:QUOTE. Может потребовать category='linear'. По умолчанию one-way mode.
Gate.io	gateio	Futures символы: BASE/QUOTE:QUOTE. settle='usdt'. API может требовать contract size.
OKX	okx	Требует passphrase кроме apiKey/secret. instType: SPOT/SWAP. tdMode: cross/isolated.

Общие нюансы

- Символы фьючерсов: BASE/QUOTE:QUOTE (USDT-margined linear perp) — единый формат ccxt
- Округление amount и price по market.precision — без этого биржа отклонит ордер
- Некоторые биржи дают precision как int (знаков после запятой), другие как float (step size) — обработать оба варианта
- load_markets() кешируется — вызывать один раз на exchange instance
- Фьючерсный инстанс биржи отдельный от спотового (defaultType: swap vs spot)
- Перед открытием шорта может потребоваться: set_leverage(), set_margin_mode()

2.5 Terminal UI (Rich)

Экраны

Экран	Содержимое	Ввод пользователя
Setup	Redis подключение, параметры	Position size (USDT), SPREAD_REDUCTION
Waiting	Анимация ожидания, канал	Нет (блокирует)
Signal	Полная инфо о сигнале + план входа	Y/N
Order Fill	Таблица статусов ордеров (live)	Нет (авто)
PnL Monitor	PnL, спред, длительность	'close' / Ctrl+C
Close	Результаты закрытия, итоговый PnL	Нет
Log	Содержимое лог-файла	Нет, переход в Waiting

Панель сигнала должна показывать

- Symbol, direction
- Buy exchange + market, buy_ask, buy_ask_qty
- Sell exchange + market, sell_bid, sell_bid_qty
- Спред в % (читаемо: напр. '2.65%' а не '1.0265')
- Возраст сигнала в секундах
- Timestamp сигнала

План входа должен показывать

- Оригинальные цены vs скорректированные
- Сдвиг цены на каждой ноге
- Оригинальный спред → спред после корректировки → разница
- Примерный amount на каждой ноге
- USDT на каждую ногу

2.6 Конкурентность и блокировка сигналов

КРИТИЧЕСКИ: Пока идёт цикл с одним сигналом (от показа до закрытия и лога), новые сигналы НЕ обрабатываются. Они копятся в **Redis**, но терминал их игнорирует до завершения текущего цикла. После завершения — берётся следующий сигнал из потока, если он ещё актуален.

- Один активный трейд в любой момент времени
- Сигналы поступают в очередь **Redis pub/sub** (буфер)
- После завершения цикла — проверить возраст следующего сигнала — если слишком старый, пропустить
- Порог актуальности: настраиваемый (**SIGNAL_MAX_AGE_SEC**, по умолчанию 30с)

2.7 Логирование

- Каждое действие логируется с **timestamp** в объект **TradeSession.log[]**
- По завершении — запись в **logs/trade_{SYMBOL}_{TIMESTAMP}.log**
- Лог содержит: сигнал, параметры, оба лега (вход + закрытие), **timeline**, все события
- Лог показывается в терминале после закрытия

2.8 Настраиваемые параметры

Параметр	Тип	Описание	Default
REDIS_HOST	str	Redis host	127.0.0.1
REDIS_PORT	int	Redis port	6379
REDIS_CHANNEL	str	Канал сигналов	ch:spread_signals
EXCHANGE_KEYS	dict	API ключи для 4 бирж	{}
POSITION_SIZE_USDT	float	Размер позиции на ногу	100.0
SPREAD_REDUCTION	float	Снижение спреда в пп (общее)	0.30
ORDER_POLL_INTERVAL	float	Интервал поллинга ордеров (сек)	1.0
PNL_POLL_INTERVAL	float	Интервал PnL обновления (сек)	2.0
SIGNAL_MAX_AGE_SEC	float	Макс возраст сигнала для актуальности	30
LOG_DIR	str	Папка логов	logs/

2.9 Обработка ошибок

Ситуация	Поведение
Один ордер ошибка, другой OK	Показать ошибку, спросить оператора: отменить успешный или продолжить с одной ногой
Оба ордера ошибка	Показать ошибки, вернуться в IDLE
Redis disconnect	Попытка reconnect с экспоненциальным backoff
Ордер не заполняется 5 мин	Спросить: отменить или ждать дальше
API rate limit	ccxt enableRateLimit=True и retry
Ошибка при закрытии	Показать, повторить попытку закрытия
Непарсимый сигнал	Пропустить, лог в stderr
Ctrl+C во время позиции	Закрыть позицию, сохранить лог, выйти

2.10 Зависимости

Пакет	Назначение
Python 3.10+	Основной язык
redis	Redis pub/sub клиент
ccxt	Унифицированный API для бирж
rich	Terminal UI (tables, panels, live display)
prompt_toolkit (опционально)	Продвинутый ввод в терминале

③ ПРОМПТ ДЛЯ РАЗРАБОТКИ

Ниже — промпт, который можно скопировать и отправить AI-ассистенту (Claude, ChatGPT ит.д.) для написания полного кода проекта.

```
# TASK
Build a terminal-based Python application for semi-automated spot↔futures
arbitrage trading.

# SIGNAL SOURCE
The app subscribes to Redis pub/sub channel `ch:spread_signals`.
Signals come in TWO formats — parser must try JSON first, then CSV on failure:

## JSON format:
```json
{
  "symbol": "THEUSDT",
  "direction": "A",
  "buy_exchange": "binance",
  "buy_market": "spot",
  "buy_ask": "0.2338",
  "buy_ask_qty": "5000",
  "sell_exchange": "gate",
  "sell_market": "futures",
  "sell_bid": "0.2362",
  "sell_bid_qty": "3000",
  "spread_pct": 1.0265,
  "ts_signal": 1773590375286
}
```

CSV format:
```
buy_exchange,sell_exchange.SYMBOL,buy_ask,sell_bid,spread_pct,ts_signal
binance,gate.THEUSDT,0.2338,0.2362,1.0265,1773590375286
```

EXCHANGES
Supported: binance, bybit, gate.io (gateio in ccxt), okx.
Use ccxt library. Each exchange needs TWO instances: spot (defaultType: spot) and
futures (defaultType: swap).
Futures symbols in ccxt: BASE/QUOTE:QUOTE (e.g. THE/USDT:USDT for USDT-margined
linear perp).
Spot symbols: BASE/QUOTE (e.g. THE/USDT).
OKX requires apiKey + secret + password (passphrase).

ENTRY PRICE LOGIC (CRITICAL — GET THIS RIGHT)

Configurable parameter: SPREAD_REDUCTION (default 0.30) — total spread reduction
in percentage points.
```

This means: if market spread is 2.8%, our limit orders create a 2.5% spread between them (2.8 - 0.30 = 2.5).
The reduction is split equally between both legs: half_reduction = SPREAD_REDUCTION / 2.

```
Formulas:
  market_spread_pct = (sell_bid / buy_ask - 1) * 100
  half_reduction = SPREAD_REDUCTION / 2  # default: 0.15
  adjusted_buy  = buy_ask  * (1 + half_reduction / 100)  # pay MORE → buy price goes UP
  adjusted_sell = sell_bid * (1 - half_reduction / 100)  # receive LESS → sell price goes
DOWN
```

Verification: (adjusted_sell / adjusted_buy - 1) * 100 ≈ market_spread_pct - SPREAD_REDUCTION

```
Example: buy_ask=0.2338, sell_bid=0.2362, SPREAD_REDUCTION=0.30
  market_spread = 1.026%, half = 0.15
  adjusted_buy  = 0.2338 * 1.0015 = 0.23415
  adjusted_sell = 0.2362 * 0.9985 = 0.23585
  result_spread = (0.23585 / 0.23415 - 1) * 100 ≈ 0.726%  (1.026 - 0.30 = 0.726 ✔)
```

The price shift is PROPORTIONAL (multiply by 1±fraction), NOT absolute addition.

APPLICATION FLOW

1. Setup Phase
- Ask operator for position_size_usdt (same for both legs, default 100)
- Ask operator for SPREAD_REDUCTION (default 0.30 pp)
- Connect to Redis
- Initialize exchange instances (lazy — only when first signal for that exchange arrives)

2. Signal Loop (IDLE state)
- Listen to Redis channel, block until signal arrives
- Show signal info panel: symbol, exchanges, prices, spread %, age
- Show entry plan: original prices, adjusted prices, amounts, spread before/after
- Ask: "Enter this trade? [Y/N]"
- If N → skip, return to listening

3. Opening (if Y)
- Place BOTH limit orders in PARALLEL (threading, not sequential!)
  - BUY leg: limit buy on spot exchange at adjusted_buy
  - SELL leg: limit sell on futures exchange at adjusted_sell (opens short)
- Round amount and price to exchange precision (market.precision.amount / .price)
  Handle both int precision (decimal places) and float precision (step size)

4. Fill Monitoring
- Poll order status every ~1 second
- Show live table: order_id, exchange, side, symbol, intended price/amount, filled amount, avg price, status
- Use Rich Live for real-time terminal updates
- Once BOTH orders are filled (status=closed) → move to PnL monitor
- Timeout 5 min: ask operator to cancel or continue waiting

5. PnL Monitor
- Fetch tickers from both exchanges every 1-2 seconds
- Calculate and show:
  - Buy PnL (long spot): (current_bid - entry_price) * filled_amount
  - Sell PnL (short futures): (entry_price - current_ask) * filled_amount
  - Total PnL
  - Entry spread vs current spread
  - Position duration
- Use Rich Live display
- Operator types 'close' or presses Ctrl+C to close

6. Closing
- Place BOTH market orders in PARALLEL:
  - BUY leg close: market SELL (sell the spot position)
  - SELL leg close: market BUY (close the futures short)
- Show final results

7. Logging
- Save complete log to logs/trade_{SYMBOL}_{YYYYMMDD_HHMMSS}.log
- Log contents: signal, parameters, both legs (entry + close), timeline, all events
- Display log in terminal
- Return to IDLE → next signal

CRITICAL BLOCKING RULE
While a trade cycle is active (from signal display to log), do NOT process new signals.
After cycle completes, take the next signal from Redis stream.
Check signal age — skip if older than SIGNAL_MAX_AGE_SEC (default 30s).

ERROR HANDLING
- One order fails, other succeeds: show error, ask operator to cancel the successful one or continue with one leg
- Both orders fail: show errors, return to IDLE
- Redis disconnect: reconnect with exponential backoff
- Order not filling 5 min: ask operator
- Close order fails: show error, retry
- Unparseable signal: skip, log to stderr
- Ctrl+C during position: close position, save log, exit cleanly

TECH STACK
- Python 3.10+
- redis (pub/sub client)
- ccxt (exchange API, enableRateLimit=True)
- rich (Terminal UI: Table, Panel, Live, Prompt, Confirm)
- threading (parallel order placement)

FILE STRUCTURE
```
arb_terminal/
├── config.py           # All settings: API keys, Redis, trading params
├── signal_listener.py  # Redis subscription + JSON/CSV parsing
```

```
├── exchange_manager.py # ccxt wrapper: 4 exchanges, spot+futures, rounding
├── trade_executor.py   # Entry price calc, open/monitor/close trade
├── main.py             # Terminal app (Rich TUI), state machine, main loop
├── test_signal.py      # Test utility: publish fake signals to Redis
└── logs/               # Trade log files
```

```
# EXCHANGE-SPECIFIC NOTES
- Binance futures may need positionSide or margin mode setup
- Bybit unified account, may need category='linear'
- Gate.io futures: settle='usdt'
- OKX: passphrase required, instType SPOT/SWAP, tdMode cross/isolated
- load_markets() should be cached per exchange instance
- Separate exchange instances for spot vs futures (different defaultType)
```

```
# IMPORTANT NOTES
- Fees and slippage are NOT factored into position sizing
- Actual USDT per leg may differ slightly due to rounding — this is expected
- The app is single-threaded main loop, only order placement uses threads
- SPREAD_REDUCTION is in percentage points (pp), NOT a multiplier
- All prices shown to operator should be human-readable (spread as '2.65%' not
'1.0265')
```